

Proof of Concept (PoC): SIEM Red-Blue Lab Data Capture

1. Overview

This Proof of Concept (PoC) documents the process of setting up a **Red-Blue Team lab environment** using two **Kali Linux Virtual Machines (VMs)** on VirtualBox to capture attack traffic for security analysis. The goal was to generate structured network traffic data that simulates real-world cybersecurity scenarios and prepare it for visualization in Observable.js.

2. Objectives

- Simulate **ICMP flood (DoS attack)** and **SYN scan (Reconnaissance attack)** in a controlled lab environment.
- Capture network traffic using tcpdump and process it for **Security Information and Event Management (SIEM) visualization**.
- Overcome networking and data handling challenges such as **router packet filtering, dataset size limits, and JSON/CSV conversion**.

3. Setup and Configuration

3.1 Environment

- **Host System:** Windows with VirtualBox
- **Virtual Machines (VMs):** Two instances of **Kali Linux (2024.4)**
- **Networking Mode:** Bridged Adapter with **Promiscuous Mode enabled** (to allow network visibility between VMs)

Problem	Cause	Solution
Kali’s network isolation	VirtualBox isolates VMs from LAN	Enabled Bridged Mode + Promiscuous Mode to allow VM-to-VM communication
DoS traffic affecting LAN users	Heavy packet flooding could slow down the network	Ran tests at midnight to avoid network disruption
hping3 --flood icmp packets dropped	Router filters excessive ICMP traffic	Used <code>-i u5000 -c 5000</code> to slow down ICMP traffic
SYN flood generated excessive packets	Flood mode creates massive TCP traffic	Used controlled SYN scan (nmap -sS -T2) instead of flood for the dataset
Dataset exceeded 10,000 packet limit	- Browsing added too much traffic - Too many nmap scans	Stopped live browsing during capture Used fewer, slower nmap scans to reduce TCP traffic

Mixed ICMP & SYN in one file	Too many packet types in one dataset	Captured ICMP and SYN separately for better organization
Never used JSON in Observable.js	Expected .pcap format issues	Converted .pcap → CSV using tshark for easy parsing in Observable.js

4. Attack Simulation and Data Capture

4.1 ICMP Flood (DoS Attack Simulation)

Objective: Simulate a Ping flood attack to observe DoS-like behavior.

Attack Command (Red Team - 192.168.1.91):

```
sudo hping3 --icmp -i u5000 -c 5000 192.168.1.89
```

Packet Capture (Blue Team - 192.168.1.89):

```
sudo tcpdump -i eth0 -w attack_icmp.pcap
```

Final Processed File: attack_icmp.csv

4.2 SYN Scan (Reconnaissance Attack)

Objective: Simulate an Nmap SYN scan used in network reconnaissance.

Attack Command (Red Team - 192.168.1.91):

```
sudo nmap -sS -T2 -p1-100 192.168.1.89
```

Packet Capture (Blue Team - 192.168.1.89):

```
sudo tcpdump -i eth0 -w attack_syn.pcap
```

Final Processed File: attack_syn.csv

4.3 SYN Flood (High-Volume Attack for Demonstration)

Objective: Show how a SYN flood overwhelms a system.

Attack Command (Red Team - 192.168.1.91):

```
sudo hping3 --flood --syn -p 80 192.168.1.89
```

Packet Capture (Blue Team - 192.168.1.89):

```
sudo tcpdump -i eth0 -w attack_flood.pcap
```

Final Processed File (For Presentation Only): attack_flood.pcap (1.5M+ packets, NOT for assignment dataset)

5. Data Processing & Conversion for Observable.js

Since Observable.js works best with structured data, .pcap files were converted to **CSV**:

```
tshark -r attack_icmp.pcap -T fields -E separator=, -e frame.time -e ip.src -e ip.dst -e ip.proto > attack_icmp.csv
```

```
tshark -r attack_syn.pcap -T fields -E separator=, -e frame.time -e ip.src -e ip.dst -e ip.proto > attack_syn.csv
```

Final Deliverables for Analysis:

- attack_icmp.csv – ICMP attack dataset
- attack_syn.csv – SYN scan dataset
- attack_flood.pcap – Kept for presentation (not included in assignment submission)

6. Key Findings & Lessons Learned

6.1 Technical Takeaways

- **SYN flood attacks generate extreme amounts of traffic** → Must be separated from structured scans.
- **Routers filter ICMP flood traffic aggressively** → Must reduce attack speed.
- **Network analysis requires strict packet count control** → Uncontrolled scanning and browsing led to dataset bloat.
- **Mixing attack & normal traffic makes the dataset realistic** → Balanced approach improves SIEM-like analysis.

6.2 Next Steps & Improvements

- Further refine packet filtering to **remove empty or redundant packets** in Observable.js.
- Test additional attack types (e.g., **DNS amplification or ARP poisoning**) to expand SIEM scenarios.
- Explore different SIEM tools (e.g., **Zeek, Suricata**) for deeper analysis beyond packet capture.

7. Conclusion

This **PoC successfully demonstrated a controlled cybersecurity attack simulation** within a VirtualBox SIEM lab environment. By overcoming key technical hurdles—such as router filtering, dataset size limitations, and VirtualBox network restrictions—this project produced **structured datasets for security visualization**. The final datasets are now ready for Observable.js processing and SIEM-like analysis.

```

kali@kali:~$
kali@kali:~$ ip s
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet ::1/128 scope host ::noipfixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:27:ae:13:06 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.91/24 brd 192.168.1.255 scope global dynamic nopfixroute eth0
        valid_lft 86375sec preferred_lft 86375sec
    inet6 2001:505:1222:a90b:129b:e6b8:c8c4ad37:64 scope global dynamic nopfixroute
        valid_lft 34680sec preferred_lft 34680sec
    inet6 fe80::82a5:54bc:dcd:156d: scope link nopfixroute
        valid_lft forever preferred_lft forever
kali@kali:~$
kali@kali:~$ ping 192.168.1.91
PING 192.168.1.91 (192.168.1.91) 56(64) bytes of data.
  0 bytes from 192.168.1.91: icmp_seq=1 ttl=64 time=0.539 ms
  0 bytes from 192.168.1.91: icmp_seq=2 ttl=64 time=0.506 ms
  0 bytes from 192.168.1.91: icmp_seq=3 ttl=64 time=0.315 ms
  0 bytes from 192.168.1.91: icmp_seq=4 ttl=64 time=0.283 ms
  0 bytes from 192.168.1.91: icmp_seq=5 ttl=64 time=0.387 ms
  0 bytes from 192.168.1.91: icmp_seq=6 ttl=64 time=0.931 ms
  0 bytes from 192.168.1.91: icmp_seq=7 ttl=64 time=0.283 ms
  0 bytes from 192.168.1.91: icmp_seq=8 ttl=64 time=0.262 ms
  0 bytes from 192.168.1.91: icmp_seq=9 ttl=64 time=0.323 ms
  0 bytes from 192.168.1.91: icmp_seq=10 ttl=64 time=0.237 ms

```

Ip a to establish address of red and blue VM

```
(kali㉿kali)-[~]
$ hping3 --flood --icmp -d 120 -p 80 192.168.1. 89
hping3: you must specify only one target host at a time

(kali㉿kali)-[~]
$ hping3 --flood --icmp -d 120 -p 80 192.168.1.89
[open_sockraw] socket(): Operation not permitted
[main] can't open raw socket

(kali㉿kali)-[~]
$ sudo hping3 --flood --icmp -d 120 -p 80 192.168.1.89
[sudo] password for kali:
HPING 192.168.1.89 (eth0 192.168.1.89): icmp mode set, 28 headers + 120 data bytes
hping in flood mode, no replies will be shown
^C
— 192.168.1.89 hping statistic —
1726163 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

Problem regarding dropping packets

```

(kali㉿kali)-[~]
$ ping -c 5 192.168.1.89
PING 192.168.1.89 (192.168.1.89) 56(84) bytes of data.
64 bytes from 192.168.1.89: icmp_seq=1 ttl=64 time=0.304 ms
64 bytes from 192.168.1.89: icmp_seq=2 ttl=64 time=0.271 ms
64 bytes from 192.168.1.89: icmp_seq=3 ttl=64 time=0.619 ms
64 bytes from 192.168.1.89: icmp_seq=4 ttl=64 time=0.306 ms
64 bytes from 192.168.1.89: icmp_seq=5 ttl=64 time=0.699 ms

--- 192.168.1.89 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4084ms
rtt min/avg/max/mdev = 0.271/0.439/0.699/0.181 ms

```

Ping suggests it's the router blocking packets and not firewall

```

(kali㉿kali)-[~]
$ sudo tcpdump -i eth0 icmp
[sudo] password for kali:
tcpdump: verbose output suppressed, use -v[... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
01:49:49.175238 IP 192.168.1.91 > 192.168.1.89: ICMP echo request, id 20252, seq 0, length 8
01:49:49.175264 IP 192.168.1.89 > 192.168.1.91: ICMP echo reply, id 20252, seq 0, length 8
01:49:50.175336 IP 192.168.1.91 > 192.168.1.89: ICMP echo request, id 20252, seq 256, length 8
01:49:50.175358 IP 192.168.1.89 > 192.168.1.91: ICMP echo reply, id 20252, seq 256, length 8
01:49:50.436982 IP 192.168.1.89 > 192.168.1.44: ICMP 192.168.1.89 udp port netbios-ns unreachable, length 86
01:49:51.176114 IP 192.168.1.91 > 192.168.1.89: ICMP echo request, id 20252, seq 512, length 8
01:49:51.176138 IP 192.168.1.89 > 192.168.1.91: ICMP echo reply, id 20252, seq 512, length 8
01:49:52.177563 IP 192.168.1.91 > 192.168.1.89: ICMP echo request, id 20252, seq 768, length 8
01:49:52.177591 IP 192.168.1.89 > 192.168.1.91: ICMP echo reply, id 20252, seq 768, length 8
01:49:53.178222 IP 192.168.1.91 > 192.168.1.89: ICMP echo request, id 20252, seq 1024, length 8
01:49:53.178250 IP 192.168.1.89 > 192.168.1.91: ICMP echo reply, id 20252, seq 1024, length 8
01:49:54.178580 IP 192.168.1.91 > 192.168.1.89: ICMP echo request, id 20252, seq 1280, length 8
01:49:54.178607 IP 192.168.1.89 > 192.168.1.91: ICMP echo reply, id 20252, seq 1280, length 8
01:49:54.334243 IP 192.168.1.91 > 192.168.1.41: ICMP 192.168.1.91 udp port netbios-ns unreachable, length 86
01:49:54.616121 IP 192.168.1.91 > 192.168.1.41: ICMP 192.168.1.91 udp port mdns unreachable, length 79
01:49:55.179211 IP 192.168.1.91 > 192.168.1.89: ICMP echo request, id 20252, seq 1536, length 8
01:49:55.179238 IP 192.168.1.89 > 192.168.1.91: ICMP echo reply, id 20252, seq 1536, length 8
01:49:55.806717 IP 192.168.1.89 > 192.168.1.44: ICMP 192.168.1.89 udp port netbios-ns unreachable, length 86
01:49:56.045167 IP 192.168.1.89 > 192.168.1.44: ICMP 192.168.1.89 udp port mdns unreachable, length 79
01:49:56.180551 IP 192.168.1.91 > 192.168.1.89: ICMP echo request, id 20252, seq 1792, length 8
01:49:56.180576 IP 192.168.1.89 > 192.168.1.91: ICMP echo reply, id 20252, seq 1792, length 8
01:49:57.181180 IP 192.168.1.91 > 192.168.1.89: ICMP echo request, id 20252, seq 2048, length 8
01:49:57.181129 IP 192.168.1.89 > 192.168.1.91: ICMP echo reply, id 20252, seq 2048, length 8
01:49:58.184883 IP 192.168.1.91 > 192.168.1.89: ICMP echo request, id 20252, seq 2304, length 8
01:49:58.184985 IP 192.168.1.89 > 192.168.1.91: ICMP echo reply, id 20252, seq 2304, length 8
^C
25 packets captured
25 packets received by filter
0 packets dropped by kernel

```

```

(kali㉿kali)-[~]
$ sudo hping3 -c 10 --icmp 192.168.1.89
[sudo] password for kali:
HPING 192.168.1.89 (eth0 192.168.1.89): icmp mode set, 28 headers + 0 data bytes
len=46 ip=192.168.1.89 ttl=64 id=58319 icmp_seq=0 rtt=7.7 ms
len=46 ip=192.168.1.89 ttl=64 id=58353 icmp_seq=1 rtt=11.6 ms
len=46 ip=192.168.1.89 ttl=64 id=58551 icmp_seq=2 rtt=7.1 ms
len=46 ip=192.168.1.89 ttl=64 id=58634 icmp_seq=3 rtt=2.2 ms
len=46 ip=192.168.1.89 ttl=64 id=58833 icmp_seq=4 rtt=5.2 ms
len=46 ip=192.168.1.89 ttl=64 id=59075 icmp_seq=5 rtt=5.1 ms
len=46 ip=192.168.1.89 ttl=64 id=59141 icmp_seq=6 rtt=8.8 ms
len=46 ip=192.168.1.89 ttl=64 id=59142 icmp_seq=7 rtt=10.7 ms
len=46 ip=192.168.1.89 ttl=64 id=59348 icmp_seq=8 rtt=8.3 ms
len=46 ip=192.168.1.89 ttl=64 id=59309 icmp_seq=9 rtt=6.7 ms

--- 192.168.1.89 hping statistic ---
10 packets transmitted, 10 packets received, 0% packet loss
round-trip min/avg/max = 2.2/7.2/11.6 ms
(kali㉿kali)-[~]

```

--flood causes the issue, c10 packet is received properly

```

[sudo] password for kali:
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C148080 packets captured
148084 packets received by filter
0 packets dropped by kernel

```

```

(kali㉿kali)-[~]
$ sudo tcpdump -i eth0 -w attack_test.pcap
[sudo] password for kali:
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C208242 packets captured
208248 packets received by filter
0 packets dropped by kernel
(kali㉿kali)-[~]
$

```

```

(kali@kali)-[~]
$ tshark -r attack_test.pcap -q -z io,phs
File System:
Protocol Hierarchy Statistics
Filter:

eth frames:148080 bytes:12058912
  ip frames:144321 bytes:8293807
    udp frames:356 bytes:90512
      llmnr frames:150 bytes:12758
        quic frames:41 bytes:18941
          quic frames:4 bytes:2996
            quic frames:2 bytes:2584
              nbns frames:15 bytes:1380
                mdns frames:17 bytes:2258
                  ssdp frames:125 bytes:53981
                    dns frames:4 bytes:390
                      data frames:4 bytes:804
                        tcp frames:133924 bytes:7688279
                          tls frames:269 bytes:41459
                            tcp.segments frames:1 bytes:1568
                              tls frames:1 bytes:1568
                                ssh frames:70 bytes:28970
                                  icmp frames:10029 bytes:514296
                                    nbns frames:11 bytes:1320
                                      mdns frames:6 bytes:678
                                        data frames:4 bytes:882
                                          igmp frames:12 bytes:720
                                            arp frames:279 bytes:16272
                                              ipv6 frames:3479 bytes:3748765
                                                udp frames:3383 bytes:3740465
                                                  data frames:3300 bytes:3705616
                                                    dns frames:24 bytes:3292
                                                      mdns frames:6 bytes:1345
                                                        quic frames:53 bytes:30212
                                                          quic frames:2 bytes:724
                                                            tcp frames:41 bytes:3618
                                                              dns frames:6 bytes:910
                                                                tcp.segments frames:3 bytes:351
                                                                  tls frames:2 bytes:188
                                                                    icmpv6 frames:55 bytes:4682
                                                                      data frames:1 bytes:68

```

Problem with the excessive large capture size is caused by tcp frames, which is caused by nmap -T4 scans


```
(kali㉿kali)-[~]
└─$ capinfos attack_syn.pcap
File name:          attack_syn.pcap
File type:          Wireshark/tcpdump/... - pcap
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit:  file hdr: 262144 bytes
Number of packets:  10 k
File size:          9,761 kB
Data size:          9,591 kB
Capture duration:   92.997865 seconds
Earliest packet time: 2025-03-08 02:51:19.125802
Latest packet time:  2025-03-08 02:52:52.123667
Data byte rate:     103 kBps
Data bit rate:      825 kbps
Average packet size: 897.70 bytes
Average packet rate: 114 packets/s
SHA256:             d44f9d6e768e5149b79599c5630491d8b34a8600a2467e66a1ec4755887585f3
SHA1:               45e281d93a82aa3facf76b0ff6acae33fb49f420
Strict time order:  False
Number of interfaces in file: 1
Interface #0 info:
    Encapsulation = Ethernet (1 - ether)
    Capture length = 262144
    Time precision = microseconds (6)
    Time ticks per second = 1000000
    Number of stat entries = 0
    Number of packets = 10684
```

Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshot.png

```
(kali㉿kali)-[~]
└─$ sudo tcpdump -i eth0 -w attack_test.pcap
[sudo] password for kali:
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C14232 packets captured
14241 packets received by filter
0 packets dropped by kernel

(kali㉿kali)-[~]
└─$ sudo tcpdump -i eth0 -w attack_icmp.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C21294 packets captured
21485 packets received by filter
0 packets dropped by kernel

(kali㉿kali)-[~]
└─$ sudo tcpdump -i eth0 -w attack_syn.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C10684 packets captured
10685 packets received by filter
0 packets dropped by kernel

(kali㉿kali)-[~]
└─$ sudo tcpdump -i eth0 -w attack_flood.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C1538120 packets captured
1539293 packets received by filter
0 packets dropped by kernel
```

```
(kali@kali)-[~]
$ capinfos attack_icmp.pcap
File name:          attack_icmp.pcap
File type:          Wireshark/tcpdump/... - pcap
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit:  file hdr: 262144 bytes
Number of packets:  21 k
File size:          11 MB
Data size:          11 MB
Capture duration:   47.788650 seconds
Earliest packet time: 2025-03-08 02:50:09.288804
Latest packet time:  2025-03-08 02:50:57.077454
Data byte rate:     241 kBps
Data bit rate:      1,935 kbps
Average packet size: 542.90 bytes
Average packet rate: 445 packets/s
SHA256:             1aff4bc77a93c5f459efb34ccb86d0e1fc3d081e233ac0996cc770e0fe7c4d5c
SHA1:               168867a8eca48f42de4b7d9a0322649d0dab1534
Strict time order:  False
Number of interfaces in file: 1
Interface #0 info:
    Encapsulation = Ethernet (1 - ether)
    Capture length = 262144
    Time precision = microseconds (6)
    Time ticks per second = 1000000
    Number of stat entries = 0
    Number of packets = 21294
```

Deliverables are attack_icmp.csv and attack_syn.csv as they fit the size requirements of the assignment.